Task A1 – Design a File Format: Transport Office Bus-Pass Records (7 Marks, K4)

Part A asks for design, pseudocode and reasoning, so the answer is in pseudocode and tables. The Java in the *Alternative* sections is only for illustration. Items marked `<fill>` need your own observations.

A1.1 – Represent the record

Fields in order (7): `passId, regNo, name, routeNo, boardingStop, validUntil, amount`.

(a) Delimited text file (one record per line)

```
passId,regNo,name,routeNo,boardingStop,validUntil,amount
P1001,192311001,Arun Kumar,12,Main Gate,2026-12-31,4500.00
P1002,192311002,"Divya, S.",7,Library Stop,2026-12-31,3800.00
```

Item	Decision
Field order	passId, regNo, name, routeNo, boardingStop, validUntil (ISO date `yyyy-MM-dd`), amount (decimal with 2 places)
Delimiter	comma `,`; one record per line, ended by `\n`
Name containing the delimiter	The field is wrapped in double quotes, and a quote inside a field is written twice (`"` → `""`). Example: `"Divya, S."`. (Alternative: choose a delimiter such as `
Header line	optional first line with field names; readers skip it
Parsing rule	a line is valid only if it has exactly 7 fields after quote-aware splitting

(b) Binary file written with `DataOutputStream`

```
writeUTF(passId) writeUTF(regNo) writeUTF(name) writeInt(routeNo)
writeUTF(boardingStop) writeLong(validUntilEpochDay)
writeDouble(amount)
```

Item	Decision
Field order	same as above; the reader must call `readUTF/readInt/readLong/readDouble` in exactly the same order
Field sizes	`writeUTF`: 2-byte length + UTF-8

Item	Decision
	bytes (variable); `int` = 4 bytes; `long` = 8 bytes; `double` = 8 bytes
Name containing a delimiter	No delimiter is used, because each string carries its own length prefix, so commas in names are harmless
Trade-off	compact and fast, but not human-readable and one wrong byte makes every later field unreadable
Money	for real money `BigDecimal` text or an integer number of paise is safer than `double`

Demonstration (sandbox run of `BinaryPass`): the record `P1001,192311001,"Arun, S.",12,Main Gate,20800,4500.0` is written and read back unchanged, file size 59 bytes.

A1.2 – Operations in pseudocode (text-file design)

Stream classes chosen and why

- `FileWriter` wrapped in `BufferedWriter` (opened in append mode) for adding: character stream, because the file is text. The buffer avoids a system call per `write`.
- `FileReader` wrapped in `BufferedReader` for find and list: `readLine()` is available only on `BufferedReader`, and buffering makes reading 8,000 lines fast.
- `FileInputStream`/`FileOutputStream` alone are *not* chosen: they are byte streams with no line support and no buffering.

```
PROCEDURE addPass(file, passId, regNo, name, routeNo, stop,
validUntil, amount)
    IF any field is invalid THEN raise InvalidDataException
    line ← join( quote(passId), quote(regNo), quote(name), routeNo,
quote(stop), validUntil, amount )
    OPEN writer ← BufferedWriter( FileWriter(file, append = true) )
    TRY
        writer.write(line) ; writer.newLine() ; writer.flush()
    FINALLY
        writer.close()
END PROCEDURE

FUNCTION findPass(file, passId) RETURNS record OR null
    OPEN reader ← BufferedReader( FileReader(file) )
    TRY
        WHILE (line ← reader.readLine()) is not null DO
            fields ← splitQuoteAware(line)
            IF fields.length ≠ 7 THEN CONTINUE              // damaged
line: skip
            IF fields[0] = passId THEN RETURN fields
        END WHILE
```

```
        RETURN null
    FINALLY
        reader.close()
END FUNCTION

FUNCTION listByRoute(file, route) RETURNS list
    result ← empty list
    OPEN reader ← BufferedReader( FileReader(file) )
    WHILE (line ← reader.readLine()) is not null DO
        fields ← splitQuoteAware(line)
        IF fields.length = 7 AND isInteger(fields[3]) AND
toInt(fields[3]) = route THEN
            result.add(fields)
    END WHILE
    CLOSE reader ; RETURN result
END FUNCTION
```

A1.3 – Design for damaged data

Problem	How the design detects it	How it is handled
Damaged line (missing fields, text in the amount)	Every line is validated: exactly 7 fields, numeric route and amount, valid date	`find` and `list` skip the line and carry on; the line number and reason are written to a log (`passes.bad`). One bad line never stops the whole read.
Missing file	`FileNotFoundException` / `NoSuchFileException` when opening	For *add*: create the file and write the header. For *find/list*: report "no passes issued yet" (empty result) instead of crashing.
Incomplete write after a power failure	The last line has no `\n` terminator or too few fields, so validation fails	Writing is done as one complete line + `newLine()` + `flush()`. On startup, if the file does not end with `\n`, the partial last line is dropped (moved to the bad-record log). For stronger safety: write to `passes.tmp`, flush, then rename atomically over the real file, so the file on disk is always either the old complete

Problem	How the design detects it	How it is handled
		one or the new complete one.

Alternative concept and complexity

`RandomAccessFile` with fixed-length records, so record *i* is at byte offset `i × recordSize` (sandbox-tested `FixedRecordStore`):

```
import java.io.*;

/** Fixed-length records: record i starts at i * RECORD_SIZE, so seek
is O(1). */
public class FixedRecordStore {
    static final int ID = 10, REG = 12, NAME = 30, RECORD_SIZE = ID +
REG + NAME + 4 /*route int*/ + 8 /*amount double*/;
    private final RandomAccessFile raf;
    FixedRecordStore(File f) throws IOException { raf = new
RandomAccessFile(f, "rw"); }

    private static void putFixed(RandomAccessFile r, String s, int
len) throws IOException {
        StringBuilder b = new StringBuilder(s);
        if (b.length() > len) b.setLength(len);
        while (b.length() < len) b.append(' ');                    //
padding = wasted space
        r.writeBytes(b.toString());
    }
    private static String getFixed(RandomAccessFile r, int len) throws
IOException {
        byte[] b = new byte[len]; r.readFully(b); return new
String(b).trim();
    }
    void write(int index, String id, String reg, String name, int
route, double amount) throws IOException {
        raf.seek((long) index * RECORD_SIZE);
        putFixed(raf, id, ID); putFixed(raf, reg, REG); putFixed(raf,
name, NAME);
        raf.writeInt(route); raf.writeDouble(amount);
    }
    String[] read(int index) throws IOException {                  //
O(1): one seek + one read
        raf.seek((long) index * RECORD_SIZE);
        return new String[]{getFixed(raf, ID), getFixed(raf, REG),
getFixed(raf, NAME),
                String.valueOf(raf.readInt()),
String.valueOf(raf.readDouble())};
    }
```

```
    public static void main(String[] a) throws IOException {
        File f = File.createTempFile("fixed", ".dat");
        FixedRecordStore s = new FixedRecordStore(f);
        s.write(0, "P1001", "192311001", "Arun", 12, 4500);
        s.write(1, "P1002", "192311002", "Divya, Jr.", 7, 3800);
// comma is no problem here
        System.out.println(String.join(" | ", s.read(1)) + "   record
size = " + RECORD_SIZE + " bytes");
        f.delete();
    }
}
```

Sample output: `P1002 | 192311002 | Divya, Jr. | 7 | 3800.0   record size = 64 bytes`

- **Finding a pass:** if the pass ID is the sequence number, `seek(i × size)` is **O(1)**: one seek and one read, regardless of file size. For a delimited text file you must read lines until the ID is found: **O(n)** in the number of records (n ≈ 8,000 per year; for ID lookups in an *unsorted* file the fixed-length file also needs O(n) unless the ID maps to the index, or the file is sorted and searched with binary search in O(log n)).
- **Space wasted by padding:** every field is padded to its maximum length. Example sizes: ID 10 + reg 12 + name 30 + stop 20 + date 10 + route 4 + amount 8 = 94 bytes per record. If the average real content is about 55 bytes, about 39 bytes (≈ 41 %) per record are padding. For 8,000 records: 94 × 8,000 = 752,000 bytes ≈ 735 KB, of which ≈ 300 KB is padding. That is wasted space of **O(n × (max – average length))**. Text files use only the real length plus one delimiter per field.
- **Other differences:** fixed records can be updated in place (rewrite one record), but names longer than the field are truncated and the file is not human-readable; text files are easy to inspect and edit but updates need rewriting the file.